RN-Net: Reservoir Nodes-Enabled Neuromorphic Vision Sensing Network

Sangmin Yoo, Eric Yeu-Jer Lee, Ziyu Wang, Xinxin Wang, Wei D. Lu*
Department of Electrical Engineering and Computer Science
University of Michigan
Ann arbor, MI, USA
{ysangmin, ericylee, ziwa, xinxinw, wluee}@umich.edu

Abstract

Event-based cameras are inspired by the sparse and asynchronous spike representation of the biological visual system. However, processing the event data requires either using expensive feature descriptors to transform spikes into frames, or using spiking neural networks that are expensive to train. In this work, we propose a neural network architecture, Reservoir Nodes-enabled neuromorphic vision sensing Network (RN-Net), based on simple convolution layers integrated with dynamic temporal encoding reservoirs for local and global spatiotemporal feature detection with low hardware and training costs. The RN-Net allows efficient processing of asynchronous temporal features, and achieves the highest accuracy of 99.2% for DVS128 Gesture reported to date, and one of the highest accuracy of 67.5% for DVS Lip dataset at a much smaller network size. By leveraging the internal device and circuit dynamics, asynchronous temporal feature encoding can be implemented at very low hardware cost without preprocessing and dedicated memory and arithmetic units. The use of simple DNN blocks and standard backpropagation-based training rules further reduces implementation costs.

1 Introduction

Event-based cameras are neuromorphic vision sensors that produce visual signals as asynchronous spikes.[1] An event camera produces a spike when and only when a momentary pixel intensity difference exceeds a threshold, which can offer better energy efficiency and latency when compared with conventional cameras that produce data at a constant frame rate even when the scene is stationary during video recording.

Various visual tasks have been implemented using event-based cameras. Earlier datasets were generated by reproducing conventional image classification datasets such as MNIST[2], CIFAR10[3], and Caltech100[2], where stationary images were placed in front of the camera and moved around to create pixel intensity differences along time. Many networks have been proposed for and performed well on these datasets where the object does not actually change over time.[4–15] Behaviorally dynamic datasets, such as DVS128 Gesture[16], DVS Lip[17] were later created to maximize the event camera's strength where the temporal evolutions of both the object's shape and movement are critical.

To process these behaviorally and morphologically more dynamic event datasets, deep neural networks (DNNs) consisting of convolution and fully connected layers, and spiking neural networks (SNNs) have been adopted.[4–24] Given that DNNs are specialized in processing static data, recurrent units, such as long short-term memory (LSTM)[25, 26], gated recurrent unit (GRU)[27] and bidirectional

*Corresponding author

Preprint. Under review.

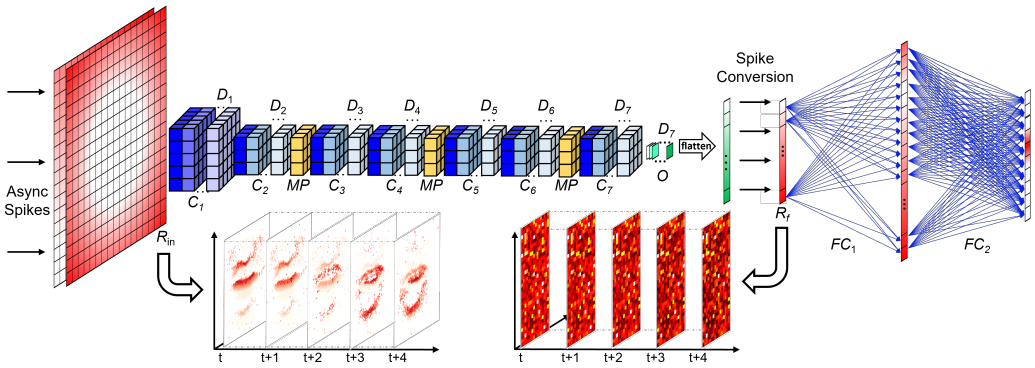

Figure 1: RN-Net structure. R_{in} and R_f are reservoir layers for local and global temporal feature encoding, respectively. Bottom left: outputs from R_{in} from a representative input in the DVS Lip dataset. Bottom right: outputs from R_f. Outputs from R_f are reshaped in 2D for better visualization. Deeper color in R_{in}, R_f and output layers of Convolution (Conv) blocks and Fully-Connected (FC_{1-2}) layers represents a higher analog value. Hidden layers within Conv blocks are not presented. The DNN structure for DVS Lip dataset is representatively illustrated. C_N, D_N, MP, FC_N represent N-th Conv layer, kernel Depth of N-th Conv layer, Max Pooling layer and N-th FC layer, respectively.

gated recurrent unit (BiGRU)[17–20] are typically required to process temporally sequential (global) information hidden in the series of momentary features. The features are processed synchronously in temporally local frames which are created either by simple accumulation of spikes within a prefixed time range[22] or using input representation algorithms like Graph construction[24, 28], 3D point cloud[21], Event frame[29], Event spike tensor[23] and voxel grid.[17, 30]. Synchronized processing requires storing and analyzing a large number of events as a pre-processing step, which diminishes the advantages of asynchronous and sparse spike generation features of event-based cameras. Recurrent units require storing multiple state data for each node and increase the training cost.

SNNs store temporal information in the neuron dynamics using models such as leaky integrate-and-fire (LIF) neurons[6, 10] and can be trained using gradient-based approaches such as backpropagation through time (BPTT).[31] To address the non-differentiability of spikes, surrogate approaches that replace spikes with neuron membrane potential have been developed.[32] However, BPTT requires backpropagation through both the network layers and through time, and is expensive to train.

To use spikes directly, feature descriptors such as Time Surface (TS)[14, 15, 33] were proposed. TS stores only the last spike event for every pixel, and converts time to the last spike information into an analog value. The analog surface after TS conversion allows the encoded data to be processed with DNN and trained using gradient-based training. However, since TS only stores the last spike, it lacks capability to handle spatiotemporal features whose correlation is beyond its temporal neighbors, which is common in real-world problems. More advanced approaches such as Leaky Surface were subsequently developed to encode the temporal information beyond the last spike[34]. However, expensive pre-processing is still required, for example, to store the previous node state and time elapsed from the latest spike, and to calculate the new state for every pixel at every time instant.

In this paper, we introduce a neural network architecture, Reservoir Nodes-enabled neuromorphic vision sensing Network (RN-Net) (Figure 1), that uses simple reservoir node (RN) layers along with DNN blocks for efficient processing of asynchronous data with low hardware implementation and training costs. Two RN layers are employed, with the one at the front natively encoding the temporally local information of asynchronous input spikes (events) for compatibility with DNN process, and the one at the back encoding the temporally global information for final classification, respectively. Physical implementations of RN-Net can be achieved with simple DNN blocks and RNs based on short-term memory (STM) memristors.[35, 36] Specifically, the RNs allow native feature description of the temporal information in all prior spike inputs produced by the event-based camera or hidden layers, without the use of recurrent units or dedicated memory and logic circuits to implement complex feature descriptor algorithms.[35–38] In this regard, RN-Net can be trained with simple DNN backpropogation rules, while maintaining better temporal information than TS encoding at lower hardware cost. RNs in the network can be thought of as analogous to cells in a retina which directly sense and encode raw and asynchronous visual inputs, and transmit them to the brain.[39] An example of the proposed RN-Net is shown in Figure 1.

The main contributions of this work can be summarized below:

• We implement a neural network structure that can directly process asynchronous spatiotemporal data generated by event-based cameras.

• RNs based on STM memristors allow efficient temporal spike encoding at lower cost. The use of simple DNN blocks allows efficient gradient-based learning rules.

• RN-Net achieves the highest reported accuracy of 99.2% on DVS128 Gesture, and one of the highest classification accuracies of 67.5% on DVS Lip dataset with a much lighter network structure than the previously reported networks.

2 Background

2.1 Event-Based Dataset

Beyond converting conventional static visual task datasets using the event-based camera[2, 3], dedicated datasets have been created to maximize the strengths of event-based cameras.[16, 17, 40–43] Among them, we use the DVS128 Gesture and DVS Lip datasets to test our model. The DVS128 Gesture dataset is for human action recognition and the DVS Lip dataset is for word recognition based on lip motions of speaking participants. Both datasets include temporally dynamic data in terms of both shape and movement. DVS128 Gesture data are labeled in 11 categories (10 designated actions and 1 random action), while DVS Lip data are labeled in 100 categories with 50 relatively easy words and 50 relatively hard words. The number of training/test data of DVS128 Gesture and DVS Lip dataset are 1077/264 and 14896/4975, respectively.

2.2 Related Work

There are various approaches to process event (spike) inputs generated either from the event camera or from neurons in the preceding layers. SNNs rely on neuronal dynamics and can theoretically process temporal data in an asynchronous fashion. However, BPTT training is expensive, and local training rules result in loss of accuracy. DNN-based networks lack neuronal dynamics and require the spiking inputs (events) to be converted to analog static frames using various feature descriptor techniques to encode the temporal information. The simplest, and often used, method, is to neglect temporal information in the spike stream and just accumulate the temporally neighboring spikes to build frames as inputs.[22] More sophisticated input representation techniques including Point clouds[21], Graph construction[24, 28], Grid-like[23], Image-like[22] and Voxel grid[30] have been proposed to convert the temporally local spike streams into frames for DNN processing.[30–32] However, these proposed feature descriptors require extensive preprocessing including random[21] or local density-based[24] down-sampling, time-scaling[17] and dedicated memory and arithmetic units to store the spatiotemporal information of the spikes and to perform the signal conversion. These costs diminish the latency and power advantages of event-based cameras, and the preprocessing often needs to be performed offline and prevents real-time operation.

In a representative feature descriptor such as TS, the amplitude of each node is reset to 1 every time it receives a spike and then relaxes following a decay function:

$$S(t) = e^{-\frac{t-T(t)}{\tau}} \tag{1}$$

where *S(t)* is an analog vector representing a node on the time surface, *t* is the current time, *T(t)* is the time information of the last spike received by the node, and τ is a pre-defined time constant.[33, 44] Since only the last spike instant needs to be recorded, this encoding method reduces memory and computing cost compared to other feature descriptors. However, useful information prior to the last spike, which may be essential for features that evolve over longer history, is discarded. Additionally, storing the last spike timing information for each pixel (node), and calculating the analog state based on equation (1) still incurs substantial costs. Other techniques such as Leaky Surface were proposed to resolve the limitation of the last-spike-dominant encoding, but require higher hardware overhead to keep track of the state and time elapsed for all nodes.[34]

2.3 Reservoir Nodes

We note that in a reservoir computing (RC) system, the reservoir maintains short-term memory and performs nonlinear transformation (encoding) of the temporal input data into the reservoir states, represented by the states of the reservoir nodes (RNs). RC systems have been efficiently implemented in hardware using devices such as memristors for vision (MNIST handwritten digits recognition), speech (NIST TI46) and Time-series forecasting (Mackey-Glass time series) tasks.[35, 36] In these systems, the reservoir nodes encode the spikes' spatiotemporal information naturally following the internal device dynamics, without any external memory or arithmetic and logic units (ALUs). By leveraging the internal device and circuit dynamics to process temporal data, these implementations have shown excellent energy efficiency and performance.[35, 36, 45–47]

Generally, due to the STM property, RNs will be more strongly affected by the near history events and weakly affected by far history events, with the extent of the nonlinearity determined by the internal RN time constant.[38, 48] Inspired by this principle, we hypothesize that RNs can be directly used to encode temporal spike data similar to what TS aims to accomplish, but at a lower cost and performing better spatiotemporal feature extraction beyond the last spike since RNs *nonlinearly* accumulate all incoming spike information.

3 Method

The mechanism of RN encoding will be discussed in Section 3.1. The RN-Net architecture and training method will be introduced in Section 3.2 and Section 3.3, respectively.

3.1 Event Encoding with Reservoir Nodes

A reservoir node can be implemented using only a single STM memristor, making hardware implementation very light weight.[35, 36] In a STM memristor, the node state (i.e. device conductance) is natively excited by the incoming spikes and relaxes in between the spikes,[38, 48] as described by the following equation:

$$G_t = P_c * (G_{max} - G_{t-1}) * \delta_{spk}(t) + G_{t-1} * e^{-\frac{1}{\tau}} \tag{2}$$

where G_t is the device (node) state at time t, P_c is a potentiation factor, G_{max} is the upper bound of the device state, τ is the characteristic relaxation time constant, and $\delta_{spk}(t)$ is a delta function representing a spiking event:

$$\delta_{spk}(t) = \begin{cases} 0, & \text{when no spike.} \\ 1, & \text{when receiving spike.} \end{cases} \tag{3}$$

$\delta_{spk}(t)$ can represent incoming events from an event-based camera or spikes from preceding layers within the network.

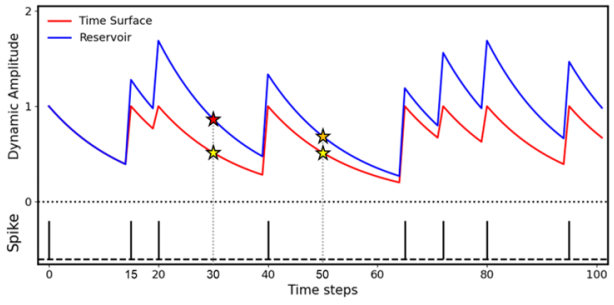

Figure 2: Dynamics of a reservoir node (blue) and a time surface node (red) under identical spikes (black) shown below.

Figure 2 shows the comparison of the RN approach implemented with a single STM memristor based on equation (2) versus the TS approach introduced in Section 2.2. Both approaches convert the spiking patterns into an analog state that can be processed by subsequent DNN blocks. Compared to TS encoding that resets the state to 1 after each spike, the RN implementation allows longer term history to be represented in the state in a non-linear fashion. For example, at t=50, the TS output is identical to that at t=30, since both values were reset to 1 10 time steps earlier (at t=20 and 40, respectively).

However, this representation missed the differences in the two temporal sequences (e.g. an additional preceding spike at t=15). On the other hand, the RN implementation clearly differentiates the two cases as all inputs prior to the current time are accumulated nonlinearly. Combined with the hardware efficiency, e.g., equation (2) is natively implemented in a single device owing to internal device physics (ionic and electronic dynamic processes)[35, 36] without the need of additional dedicated memory (to store t and $T(t)$) and ALUs to convert the temporal data, we believe the RN-Net to be a suitable solution for asynchronous, real-time processing of event data.

The proposed RN-Net operates by directly taking asynchronous spikes as they are generated, and the memristor-based RNs transform the temporal spikes into analog values following equation (2) in real time. The conductance values from the RNs are retrieved only when the network operates a forward-pass, as shown in Figure 1, analogous to visual inputs encoded by cells in a retina.[39]

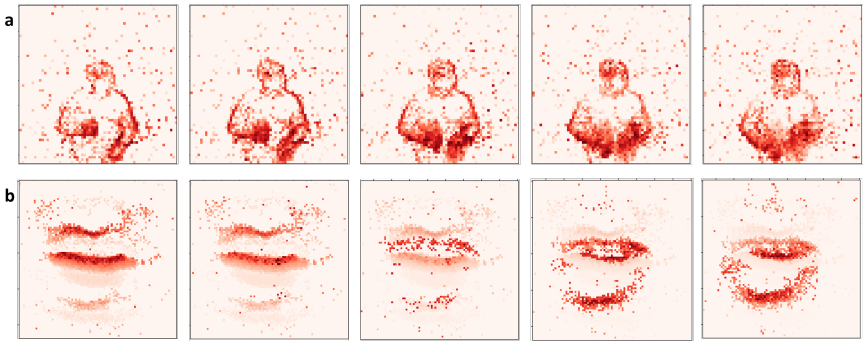

Figure 3: Temporally consecutive states of the input reservoir nodes, responding to asynchronous events in the (a) DVS128 Gesture dataset and (b) DVS Lip dataset. Each state is retrieved at a constant time interval of 30ms. Deeper red represents higher amplitude value of a RN state.

Figure 3 shows examples of 5 temporally consecutive states of the input layer (R_{in}) RNs, responding to asynchronous events from the two datasets, respectively. The states are obtained at a constant time interval (i.e. 30ms). As shown in the figure, frequent input spikes lead to stronger RN states due to the excitation term (the first term of equation (2)), while RN states natively relax for inputs that are temporally too far from the current time instant due to the internal decay term (the second term of equation (2)). These properties of the RNs allow the R_{in} layer to capture both the spatial and temporal features of the inputs, as shown in Figure 3. For example, in Figure 3b, both the speaking motion and movement (top to bottom) of the lips can be captured by the RNs' states in the R_{in} layer. Temporally fast movements are reflected in a single moment (e.g. the 3rd plot in Figure 3b), while temporally slow movements are reflected in multiple moments captured at the different time instants (e.g. the first two plots).

3.2 Model Architecture

The RN-Net is formed sequentially by an input RN layer (R_{in}) as a feature descriptor for temporally local encoding, convolution (Conv) layers (C_N), a spike conversion (SC) layer, another RN layer (R_f) for temporally global encoding, and fully-connected (FC) layers (F_N) for classification. Figure 1 illustrates the overall architecture of the proposed network for the DVS Lip dataset.

The R_{in} layer has as many RNs as the output dimension of the event-based camera. Each RN in the R_{in} layer processes input spikes asynchronously from a corresponding pixel in the event camera in real time without any preprocessing, following equation (2). Examples of the outputs generated by R_{in} are shown in Figure 3.

Depending on the application, different temporal resolutions can be chosen by adjusting the potentiation factor P_c and the time constant τ of equation (2), and the interval of R_{in} state acquisitions. A shorter acquisition interval creates more frequent activations for the following layers. A shorter time constant τ and more frequent acquisitions produce temporally finer outputs, while a longer time constant and less frequent acquisitions produce spatially more detailed outputs owing to more input spikes and the slower internal decay.[17] The convolution layers (C_1-C_7 with depths D_1-D_7 in Figure 1) then process the encoded spatiotemporal features in R_{in} at a constant time interval (i.e. 30ms).

Similar to Conv blocks in conventional DNNs, the output O from the conv layers reflects the spatial features existing in the R_{in} states, and in this case, capturing the spatiotemporal features from the original spikes.

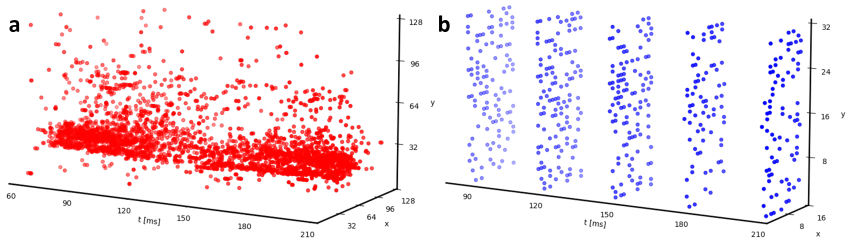

Figure 4: (a) Visualization of asynchronous input spikes and (b) spikes generated after the spike conversion (SC) layer.

Output O is then flattened and sent to the spike conversion SC layer. We use a predetermined threshold to convert the outputs O from the Conv layers into spikes in the SC layer. Spikes generated after the SC layer are shown in Figure 4. Thus, the spikes after SC represent the local spatiotemporal features captured by the R_{in} layer and the Conv layers, and become much sparser compared with the original spiking inputs.

The spikes from the SC layer are then supplied to the RNs in the R_f layer for global spatiotemporal feature capture and subsequent classification in the FC layers. Similar to the RNs in the R_{in} layer, RNs (e.g. also implemented with STM memristors) in R_f naturally potentiates/relaxes with the presence/absence of spikes from the SC layer. The state of R_f thus represents the historical (global) spatiotemporal features by encoding the processed local spatiotemperal features over time, and is then used by the subsequent FC layers (FC_1-FC_2 in Figure 1) to perform final classification functions. Similar to the RNs in the R_{in} layer, RN states in R_f are retrieved at a certain time interval (i.e. 0.3s, which is longer than that used in R_{in}) over the whole video clip and fed to the FC layers.[49]

3.3 Training Method

We train RN-Net with standard backpropagation, using the output potentials calculated by FC_2 at the end of data presentation. Unlike BPTT that calculates error and gradient across each timestep[31], error and gradient of RN-Net is calculated only once for one input training video, leading to lower training cost. To address the non-differentiability arising from spike generation in the SC layer, we adopted surrogate gradient for the SC layer. Since surrogate is used only in one layer of the network, we expect the error introduced by this approach is lower when compared with SNN approaches which require surrogates at all layers.

We also applied several data augmentation techniques, such as random/center crop, horizontal flip and Gaussian noise to minimize overfitting effects. In the case of random/center crop, we referred to existing works.[17, 19] During training, we center-cropped the original input dimension (128 x 128) to 96 x 96 and then random cropped them to 76 x 76, while the inputs are directly center-cropped to 76 x 76 in the testing phase. Horizontal flip with a probability of 0.5 and Gaussian noise with standard deviation of $5e^{-4}$ were used during training for the same purpose. These techniques helped generalize the input data and reduce overfitting during training. These techniques were used only for the DVS Lip dataset since motions in DVS128 Gesture such as a left/right hand waving are sensitive to the horizontal flip and objects positions.

4 Experiments

The experimental setups for the two datasets are in Section 4.1, and the results are discussed in Section 4.2. In Section 4.3, we present an ablation study to show the utility of each component of the network in a controlled manner.

4.1 Experiment Setup

The DVS128 Gesture dataset consists of repetitions of the same motion of a participant along the clip, while lip motions in the DVS Lip dataset are not repetitive, as shown in Figure 3. As a result, a

fraction of a clip should be sufficient to classify the action in DVS128 Gesture, while the whole clip is necessary for DVS Lip classification. Based on this understanding, we only used the first 1.5s for all videos in the DVS128 Gesture dataset during training and inference, corresponding to 10% of the longest video (15.5s). The DVS Lip dataset has variable input lengths due to the irregular length of the words and the unique speaker's traits, with most video clips (99%) having lengths between 0.75s and 1.5s. To make the data size regular across the dataset, we chose to keep the captured R_{in} outputs at 50 by simply applying null (no spike) for clips shorter than 1.5s and clipping videos longer than 1.5s during DVS Lip training and inference. X- and Y-dimension were set as 128 and 128, respectively. P_c and τ in equation (2) were set to 0.5 and 60ms.

For both datasets, the same potentiation factor P_c (0.5) and time constant τ (60ms) in equation (2) are used for the RNs in R_{in}. The states of R_{in} are captured every 30ms and sent to the subsequent Conv layers.

Due to the different input dimensions and the different number of categories in the two datasets, the convolution layers and FC layers in RN-Net are configured accordingly. The detailed network configurations for both datasets are summarized in Table 1.

Table 1: Network configuration of RN-Net for DVS128 Gesture dataset (left columns) and DVS Lip dataset (right columns).

DVS128 Gesture					DVS128 Lip				
Layer	Kernel Size	Out channel	Pad/ Stride	Output Dim	Layer	Kernel Size	Out channel	Pad/ Stride	Output Dim
R_{in}	-	2	-	128 x 128	R_{in}	-	2	-	76 x 76
MaxPool	2	2	0 / 2	64 x 64	Conv1	5	64	0 / 2	36 x 36
Conv1	3	64	0 / 1	62 x 62	Conv2	3	128	1 / 1	36 x 36
MaxPool	3	64	0 / 2	30 x 30	MaxPool	3	128	0 / 2	17 x 17
Conv2	3	128	1 / 1	30 x 30	Conv3	3	128	1 / 1	17 x 17
MaxPool	3	128	0 / 2	14 x 14	Conv4	3	256	1 / 1	17 x 17
Conv3	3	256	0 / 1	12 x 12	MaxPool	3	256	0 / 2	8 x 8
Conv4	3	512	1 / 1	12 x 12	Conv5	3	256	1 / 1	8 x 8
MaxPool	3	512	0 / 2	5 x 64	Conv6	3	512	1 / 1	8 x 8
Conv5	3	512	0 / 1	3 x 64	MaxPool	3	512	0 / 2	3 x 3
MaxPool	3	512	0 / 1	1 x 64	Conv7	3	512	0 / 1	1 x 1
R_f	-	512	-	-	R_f	-	512	-	-
FC1	-	512	-	-	FC1	-	512	-	-
FC2	-	11	-	-	FC2	-	100	-	-

The output of a DNN block consists of a convolution and a pooling layer (if exists), following:

$$O_N = f(BN(MP(C(I_N)))) \tag{4}$$

where O_N is the output of N-th layer, *f(x)* is a ReLU activation function, *BN(x)* is a batch normalization function, *MP(x)/C(x)* is a maxpooling/convolution operation, and I_N is input of the N-th layer.[50]

After the last convolution layer, the outputs are converted to spikes for subsequent temporal feature encoding at R_f.[51] The SC layer generates spikes by comparing the analogue values in O (Figure 1) with a pre-fixed threshold value, set as 0.3 for both datasets.

RNs in the R_f layer encode the global spatiotemporal features. P_c and τ for the RNs in R_f are set as 0.1, 2s and 0.3, 2s for DVS128 Gesture and DVS Lip dataset, respectively. A longer τ is used in R_f than in R_{in} to process the longer temporal correlations. The RN states in the R_f layer are captured every 0.3s for both datasets (corresponding to 5 R_f acquisitions during a 1.5s clip). In the case of clips shorter than 1.5s, we let RNs in R_f continue relaxing after the last meaningful input without padding the input data with new spikes (e.g., through repeating the clip). These relaxed states are still captured following the normal schedule and fed to the classification layers. Examples of RN states at different time instants are shown in Figure 5, along with the spikes they receive from the SC layer. For better visualization, the data are reshaped in 2D format. As shown in the figure, even without any new inputs (e.g., after 0.9s for Data2), the RN states do not immediately decay to zero due to the slow decay term in equation (2), and the evolution of the RNs still represents useful information. This approach also simplifies training and inference processes and allows the system to handle inputs with irregular lengths.

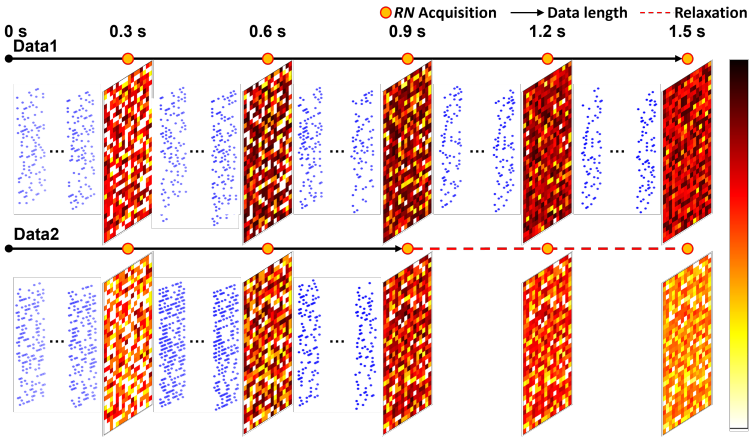

Figure 5: Visualization of outputs from R_f over the whole 1.5s clip, along with spikes generated from the spike conversion layer. For Data2 whose input video length is only 0.9s, the RN states will continue to relax and still used for classification.

A classification block consists of a FC layer, an activation function, and a batch normalization layer, following:

$$O_F = f(BN(FC(I))) \tag{5}$$

where O_F is the output of the block, *f(x)* is a ReLU activation function, *BN(x)* is a batch normalization function, *FC(x)* is an FC layer, and *I* is the input to the FC layer. In the proposed RN-Net, two classification blocks are used, and output of the 2nd block is obtained without the activation and the normalization function, and used as the final output of the network.

The Pytorch framework[52] was used for all the experiments with methods described above. SoftMax function was chosen to calculate the probability of an output neuron in final output, and the neuron with the largest probability was selected as the classification result. During training, the cost was derived by categorical cross-entropy function.[53] ATan was selected as surrogate gradient calculation for the SC layer during training.[51] We also used Adam optimizer[54] with an initial learning rate of $7e^{-3}$ / $7e^{-5}$ and a weight decay of 0 / $5e^{-3}$ as the model optimizer and ReduceLROnPlateau with a factor of 0.9 / 0.9, patience of 2 / 1, threshold of $1e^{-5}$ / $1e^{-6}$, minimum learning rate of $1e^{-4}$ / $1e^{-7}$ as the learning scheduler for DVS128 Gesture / DVS Lip, respectively. Batch size of 32 and 150 epochs were used for both datasets. All experiments are performed on an Intel Xeon Gold 6226R and an NVIDIA A40.

4.2 Experimental Results

RN-Net shows excellent performance on both datasets; 99.2% for DVS128 Gesture (highest reported to date) and 67.5% for DVS Lip (one of the highest for event-inputs). Comparisons of classification accuracy with existing networks are shown in Table 2 and Table 3. The accuracies of networks listed in Table 2 were obtained from [17]. For DVS128 Gesture, RN-Net outperforms all existing networks based on SNN and CNN platforms, while only using the first 10 percent of the longest clip. For DVS Lip, RN-Net achieves top 2 accuracy among event-based models, only behind MSTP[17] which is a multi-branch network with different input channels generating frames in different temporal granularity, and employs Voxel Grid as the feature descriptor and using larger BiGRU and ResNet-18 network architectures. By comparison, RN-Net uses a single branch and a much lighter DNN network, without expensive feature description/preprocessing and recurrent units that can lead to significant hardware and latency overheads. The relevant model sizes and their performances are listed in Table 2. The parameters for ResNet variants are obtained from [55]. Power of a proposed hardware system running RN-Net for both tasks is estimated based on existing device technology and reported values.[48] When running DVS128 Gesture and DVS Lip tasks, the proposed RN-Net hardware is estimated to consume 10.2mW and 11.4mW, respectively. Detailed explanation and methodology of the power estimation can be found in Supplementary information.

Table 2: Comparison of existing models and RN-Net on Lip reading dataset including DVS Lip.

Model	Input	ACC [%]	Params [M]	Base model	Preprocess	Local encoding	Global encoding
TANet[56]	Video	68.7	42.8	ResNet-101	-	TAM	Temporal AvgPool
ACTION-Net[57]	Video	68.8	28.1	ResNet-50	Random Sampling	STE/CE/ME	Temporal AvgPool
DFTN[18]	Video	63.2	40.5	ResNet-18	-	DFN	BiGRU
Feng et al[19]	Video	63.4	11.2	ResNet-18	-	-	BiGRU
Martinez et al[20]	Video	65.5	11.2	ResNet-18	Greyscaling	-	Multi-scale TCN
Event Clouds[21]	Event	42.2	8.1	PointNet	Random Sampling	3D point cloud	3D point cloud
EV-Gait-3DGraph[24]	Event	32.0	7.2	-	OctreeGrid Filtering	3D-Graph	N/A
EV-Gait-IMG[22]	Event	34.5	64.6	-	Event Noise Cancelling	Image-like	N/A
EST[23]	Event	48.7	21.5	ResNet-34	Normalized Time Stamp	MLP (Grid)	N/A
MSTP[17]	Event	72.1	38.5	ResNet-18	Multiple Time-Scaling	Voxel Grid	BiGRU
RN-Net	**Event**	**67.5**	**7.5**	**-**	**-**	**Reservoir**	**Reservoir**

Table 3: Comparison of existing models and RN-Net for DVS128 Gesture.

Model	Base	ACC[%]
Rollout[12]	SNN	95.7
DECOLLE[11]	SNN	95.5
SEW-ResNet[7]	SNN	97.9
tdBN[9]	SNN	96.9
PLIF[8]	SNN	97.6
LIAF-Net[10]	SNN	97.6
TA-SNN[6]	SNN	98.6
TCJA-SNN[4]	SNN	99.0
Amir et al[16]	DNN	94.6
Event Clouds[21]	DNN	95.3
RN-Net	**DNN**	**99.2**

Table 4: Performance comparison of different R_{in}/R_f configurations for DVS128 Gesture and DVS Lip tasks.

R_{in}	R_f	DVS128 Gesture	DVS Lip
TS	TS	96.2	44.5
TS	RN	97.7	49.7
RN	TS	97.0	61.0
RN	**RN**	**99.2**	**67.5**
RN	TAP	96.6	62.5

4.3 Ablation Study

The effects of the RN in R_{in} and R_f layers on model performance are investigated in an ablation study by replacing each RN layer with a TS or Temporal average pooling (TAP, only for R_f) layer without modifying the rest of the network. (Table 4) In both tasks, the case with both RN layers shows the best performance, while the case with TS at both places shows the worst performance. However, replacing R_f with TS degrades the performance more than that of R_f for the DVS128 Gesture task, while the DVS Lip task shows the opposite tendency. We attribute this difference to different characteristics of the datasets. Data in DVS128 Gesture are repetitive along the clip, which can tolerate less precise input encoding because there are opportunities to capture the lost information at a different time instant. On the other hand, in DVS Lip information at different time instants is unique, which makes input encoding more critical for the network performance. Interestingly, although the use of RNs achieves the best results for both datasets, the use of TAP in the R_f layer achieves better results than the use of TS for the DVS Lip task, while the use of TS is better for the DVS128 Gesture task. We speculate that this is caused by the characteristics of TS, which completely discards far history that is critical for the DVS Lip dataset. On the other hand, the repetitive inputs in DVS128 Gesture alleviates this problem for TS implementation, where its capability to capture local temporal dynamics outperforms TAP. As a result, this study illustrates that the RNs' capability to capture both local and global temporal dynamics allows networks based on them (i.e. RN-Net) to achieve higher accuracy for both tasks.

5 Conclusion

In this work, we propose a hybrid reservoir/DNN network, Reservoir Nodes-enabled neuromorphic vision sensing Network (RN-Net), for asynchronous event-based video processing. The use of dynamic reservoir nodes enables efficient spatiotemporal encoding of both local and global features at different hierarchies in real time and eliminates external memory and logic/recurrent units. RN-Net achieves the highest DVS128 Gesture classification accuracy, and one of the highest DVS Lip classification accuracies with a much lighter network structure. The reservoir nodes can be efficiently implemented with STM memristors, taking advantage of internal device physics to perform

signal processing. The low hardware and training costs of RN-Net make it an attractive option for event-camera applications.

Acknowledgments and Disclosure of Funding

This work was supported in part by the National Science Foundation through Awards ♯ 1915550 and ♯ CCF- 1900675, and SRC and DARPA through the Applications Driving Architectures (ADA) Research Center

References

[1] Guillermo Gallego, Tobi Delbruck, Garrick Orchard, Chiara Bartolozzi, Brian Taba, Andrea Censi, Stefan Leutenegger, Andrew J. Davison, Jorg Conradt, Kostas Daniilidis, and Davide Scaramuzza. Event-based vision: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44:154–180, 2022.

[2] Garrick Orchard, Ajinkya Jayawant, Gregory K. Cohen, and Nitish Thakor. Converting static image datasets to spiking neuromorphic datasets using saccades. *Frontiers in Neuroscience*, 9, 2015.

[3] Hongmin Li, Hanchao Liu, Xiangyang Ji, Guoqi Li, and Luping Shi. CIFAR10-DVS: An event-stream dataset for object classification. *Frontiers in Neuroscience*, 11, 2017.

[4] Rui-Jie Zhu, Qihang Zhao, Tianjing Zhang, Haoyu Deng, Yule Duan, Malu Zhang, and Liang-Jian Deng. TCJA-SNN: Temporal-channel joint attention for spiking neural networks. 6 2022.

[5] Shikuang Deng, Yuhang Li, Shanghang Zhang, and Shi Gu. Temporal efficient training of spiking neural network via gradient re-weighting. *International Conference on Learning Representations (ICLR)*, pages 1–17, 2022.

[6] Man Yao, Huanhuan Gao, Guangshe Zhao, Dingheng Wang, Yihan Lin, Zhaoxu Yang, and Guoqi Li. Temporal-wise attention spiking neural networks for event streams classification. 7 2021.

[7] Wei Fang, Zhaofei Yu, Yanqi Chen, Tiejun Huang, Timothee Masquelier, and Yonghong Tian. Deep residual learning in spiking neural networks. *Advances in Neural Information Processing Systems*, 25:21056–21069, 2021.

[8] Wei Fang, Zhaofei Yu, Yanqi Chen, Timothée Masquelier, Tiejun Huang, and Yonghong Tian. Incorporating learnable membrane time constant to enhance learning of spiking neural networks. *Proceedings of the IEEE International Conference on Computer Vision*, 1:2641–2651, 2021.

[9] Hanle Zheng, Yujie Wu, Lei Deng, Yifan Hu, and Guoqi Li. Going deeper with directly-trained larger spiking neural networks. *35th AAAI Conference on Artificial Intelligence, AAAI 2021*, 12B:11062–11070, 2021.

[10] Zhenzhi Wu, Hehui Zhang, Yihan Lin, Guoqi Li, Meng Wang, and Ye Tang. LIAF-Net: Leaky integrate and analog fire network for lightweight and efficient spatiotemporal information processing. *IEEE Transactions on Neural Networks and Learning Systems*, 33:6249–6262, 2022.

[11] Jacques Kaiser, Hesham Mostafa, and Emre Neftci. Synaptic plasticity dynamics for deep continuous local learning (DECOLLE), 2020.

[12] Alexander Kugele, Thomas Pfeil, Michael Pfeiffer, and Elisabetta Chicca. Efficient processing of spatio-temporal data streams with spiking neural networks, 2020.

[13] Sumit Bam Shrestha and Garrick Orchard. Slayer: Spike layer error reassignment in time. *Advances in Neural Information Processing Systems*, 2018-Decem:1412–1421, 2018.

[14] Xavier Lagorce, Garrick Orchard, Francesco Galluppi, Bertram E. Shi, and Ryad B. Benosman. HOTS: A hierarchy of event-based time-surfaces for pattern recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39:1346–1359, 7 2017.

[15] Amos Sironi, Manuele Brambilla, Nicolas Bourdis, Xavier Lagorce, and Ryad Benosman. HATS: Histograms of averaged time surfaces for robust event-based object classification. 2018.

[16] Arnon Amir, Brian Taba, David Berg, Timothy Melano, Jeffrey Mckinstry, Carmelo Di Nolfo, Tapan Nayak, Alexander Andreopoulos, Guillaume Garreau, Marcela Mendoza, Jeff Kusnitz, Michael Debole, Steve Esser, Tobi Delbruck, Myron Flickner, Dharmendra Modha, U C San Diego, and Uzh eth Zurich. A low power , fully event-based gesture recognition system. pages 7243–7252, 2017.

[17] Ganchao Tan, Yang Wang, Han Han, Yang Cao, Feng Wu, and Zheng Jun Zha. Multi-grained spatio-temporal features perceived network for event-based lip-reading. *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 2022-June:20062–20071, 2022.

[18] Jingyun Xiao, Shuang Yang, Yuanhang Zhang, Shiguang Shan, and Xilin Chen. Deformation flow based two-stream network for lip reading. *Proceedings - 2020 15th IEEE International Conference on Automatic Face and Gesture Recognition, FG 2020*, pages 364–370, 2020.

[19] Dalu Feng, Shuang Yang, Shiguang Shan, and Xilin Chen. Learn an effective lip reading model without pains. pages 1–6, 2020.

[20] Brais Martinez, Pingchuan Ma, Stavros Petridis, and Maja Pantic. Lipreading using temporal convolutional networks. *ICASSP, IEEE International Conference on Acoustics, Speech and Signal Processing - Proceedings*, 2020-May:6319–6323, 2020.

[21] Qinyi Wang, Yexin Zhang, Junsong Yuan, and Yilong Lu. Space-time event clouds for gesture recognition: From rgb cameras to event cameras. *Proceedings - 2019 IEEE Winter Conference on Applications of Computer Vision, WACV 2019*, pages 1826–1835, 2019.

[22] Yanxiang Wang, Bowen Du, Yiran Shen, Kai Wu, Guangrong Zhao, Jianguo Sun, and Hongkai Wen. EV-gait: Event-based robust gait recognition using dynamic vision sensors. *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 2019-June:6351–6360, 2019.

[23] Daniel Gehrig, Antonio Loquercio, Konstantinos Derpanis, and Davide Scaramuzza. End-to-end learning of representations for asynchronous event-based data. *Proceedings of the IEEE International Conference on Computer Vision*, 2019-Octob:5632–5642, 2019.

[24] Y Wang, X Zhang, Y Shen, B Du, G Zhao, L Cui, and H Wen. Event-stream representation for human gaits identification using deep neural networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44:3436–3449, 2022.

[25] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9:1735–1780, 1997.

[26] Alex Graves. Generating sequences with recurrent neural networks. pages 1–43, 2013.

[27] Junyoung Chung, Caglar Gulcehre, KyungHyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. pages 1–9, 2014.

[28] Yin Bi, Aaron Chadha, Alhabib Abbas, Eirina Bourtsoulatze, and Yiannis Andreopoulos. Graph-based object classification for neuromorphic vision sensing. *Proceedings of the IEEE International Conference on Computer Vision*, 2019-Octob:491–501, 2019.

[29] Henri Rebecq, Timo Horstschaefer, and Davide Scaramuzza. Real-time visual-inertial odometry for event cameras using keyframe-based nonlinear optimization. *British Machine Vision Conference 2017, BMVC 2017*, 2017.

[30] Alex Zihao Zhu, Liangzhe Yuan, Kenneth Chaney, and Kostas Daniilidis. Unsupervised event-based learning of optical flow, depth and egomotion. *IEEE Computer Society Conference on Computer Vision and Pattern Recognition Workshops*, 2019-June:1694, 2019.

[31] P.J. Werbos. Backpropagation through time: What it does and how to do it, 1990.

[32] Emre O. Neftci, Hesham Mostafa, and Friedemann Zenke. Surrogate gradient learning in spiking neural networks. pages 1–25, 2019.

[33] Antoine Grimaldi, Victor Boutin, Sio hoi Ieng, and Ryad Benosman. A robust event-driven approach to always-on object recognition. *TechRxiv*, 2022.

[34] Marco Cannici, Marco Ciccone, Andrea Romanoni, and Matteo Matteucci. Asynchronous convolutional networks for object detection in neuromorphic cameras. *IEEE Computer Society Conference on Computer Vision and Pattern Recognition Workshops*, 2019-June:1656–1665, 2019.

[35] Chao Du, Fuxi Cai, Mohammed A. Zidan, Wen Ma, Seung Hwan Lee, and Wei D. Lu. Reservoir computing using dynamic memristors for temporal information processing. *Nature Communications*, 8:1–10, 2017.

[36] John Moon, Wen Ma, Jong Hoon Shin, Fuxi Cai, Chao Du, Seung Hwan Lee, and Wei D Lu. Temporal data classification and forecasting using a memristor-based reservoir computing system. *Nature Electronics*, 2:480–487, 2019.

[37] John Moon, Yuting Wu, and Wei D Lu. Hierarchical architectures in reservoir computing systems. *Neuromorphic Computing and Engineering*, 1:014006, 2021.

[38] Ting Chang, Sung hyun Jo, and Wei Lu. Short-term memory to long-term memory transition in a nanoscale memristor. *ACS Nano*, 5:7669–7676, 2011.

[39] Richard H Masland. The neuronal organization of the retina. *Neuron*, 76:266–280, 10 2012.

[40] Yin Bi, Aaron Chadha, Alhabib Abbas, Eirina Bourtsoulatze, and Yiannis Andreopoulos. Graph-based object classification for neuromorphic vision sensing. 8 2019.

[41] Elias Mueggler, Henri Rebecq, Guillermo Gallego, Tobi Delbruck, and Davide Scaramuzza. The event-camera dataset and simulator: Event-based data for pose estimation, visual odometry, and SLAM. 10 2016.

[42] Mathias Gehrig, Willem Aarents, Daniel Gehrig, and Davide Scaramuzza. DSEC: A stereo event camera dataset for driving scenarios. 3 2021.

[43] Teresa Serrano-Gotarredona and Bernabé Linares-Barranco. Poker-dvs and mnist-dvs. their history, how they were made, and other details. *Frontiers in Neuroscience*, 9, 2015.

[44] Ricardo Tapiador-Morales, Jean-Matthieu Maro, Angel Jimenez-Fernandez, Gabriel Jimenez-Moreno, Ryad Benosman, and Alejandro Linares-Barranco. Event-based gesture recognition through a hierarchy of Time-Surfaces for FPGA, 2020.

[45] J Li, C Zhao, K Hamedani, and Y Yi. Analog hardware implementation of spike-based delayed feedback reservoir computing system. pages 3439–3446, 2017.

[46] Matteo Farronato, Piergiulio Mannocci, Margherita Melegari, Saverio Ricci, Christian Monzio Compagnoni, and Daniele Ielmini. Reservoir computing with charge-trap memory based on a mos2 channel for neuromorphic engineering. *Advanced Materials*, n/a:2205381, 10 2022. https://doi.org/10.1002/adma.202205381.

[47] H Fang, B Taylor, Z Li, Z Mei, H H Li, and Q Qiu. Neuromorphic algorithm-hardware codesign for temporal pattern learning. pages 361–366, 2021.

[48] Chao Du, Wen Ma, Ting Chang, Patrick Sheridan, and Wei D. Lu. Biorealistic implementation of synaptic functions with oxide memristors through internal ionic dynamics. *Advanced Functional Materials*, 25:4290–4299, 2015.

[49] Miquel L Alomar, Miguel C Soriano, Miguel Escalona-morán, Vincent Canals, Ingo Fischer, Claudio R Mirasso, and Jose L Rosselló. Digital implementation of a single dynamical node reservoir computer. *IEEE Transactions on Circuits and Systems II: Express Briefs*, 62:977–981, 2015.

[50] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *32nd International Conference on Machine Learning, ICML 2015*, 1:448–456, 2015.

[51] Jason K Eshraghian, Xinxin Wang, Mohammed Bennamoun, Max Ward, Wei D Lu, and Gregor Lenz. Training spiking neural networks using lessons from deep learning. *arXiv*, 2022.

[52] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. volume 32, pages 8024–8035. Curran Associates, Inc., 2019.

[53] Zhilu Zhang and Mert R. Sabuncu. Generalized cross entropy loss for training deep neural networks with noisy labels. *Advances in Neural Information Processing Systems*, 2018-Decem:8778–8788, 2018.

[54] Diederik P. Kingma and Jimmy Lei Ba. Adam: A method for stochastic optimization. *3rd International Conference on Learning Representations, ICLR 2015 - Conference Track Proceedings*, pages 1–15, 2015.

[55] Mei Chee Leong, Dilip K. Prasad, Yong Tsui Lee, and Feng Lin. Semi-CNN architecture for effective spatio-temporal learning in action recognition. *Applied Sciences (Switzerland)*, 10, 2020.

[56] Zhaoyang Liu, Limin Wang, Wayne Wu, Chen Qian, and Tong Lu. TAM: Temporal adaptive module for video recognition. *Proceedings of the IEEE International Conference on Computer Vision*, pages 13688–13698, 2021.

[57] Zhengwei Wang, Qi She, and Aljosa Smolic. Action-Net: Multipath excitation for action recognition. *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, pages 13209–13218, 2021.